DATA STRUCTURES

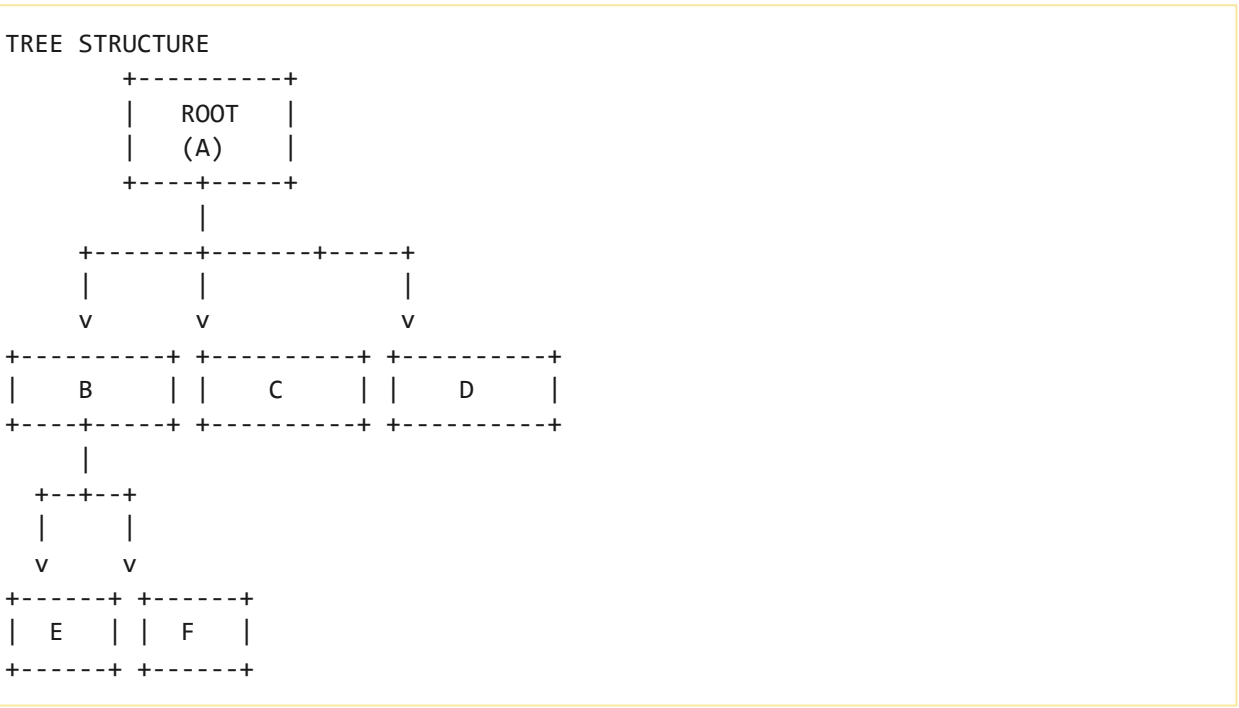
Unit-IV: Trees, Sorting & Searching

Comprehensive Study Notes

Topic 1: Tree Structures

1.1 What is a Tree?

A **tree** is a **non-linear** data structure consisting of nodes connected by edges. It has a **hierarchical** structure with a root node and child nodes.



1.2 Tree Terminology

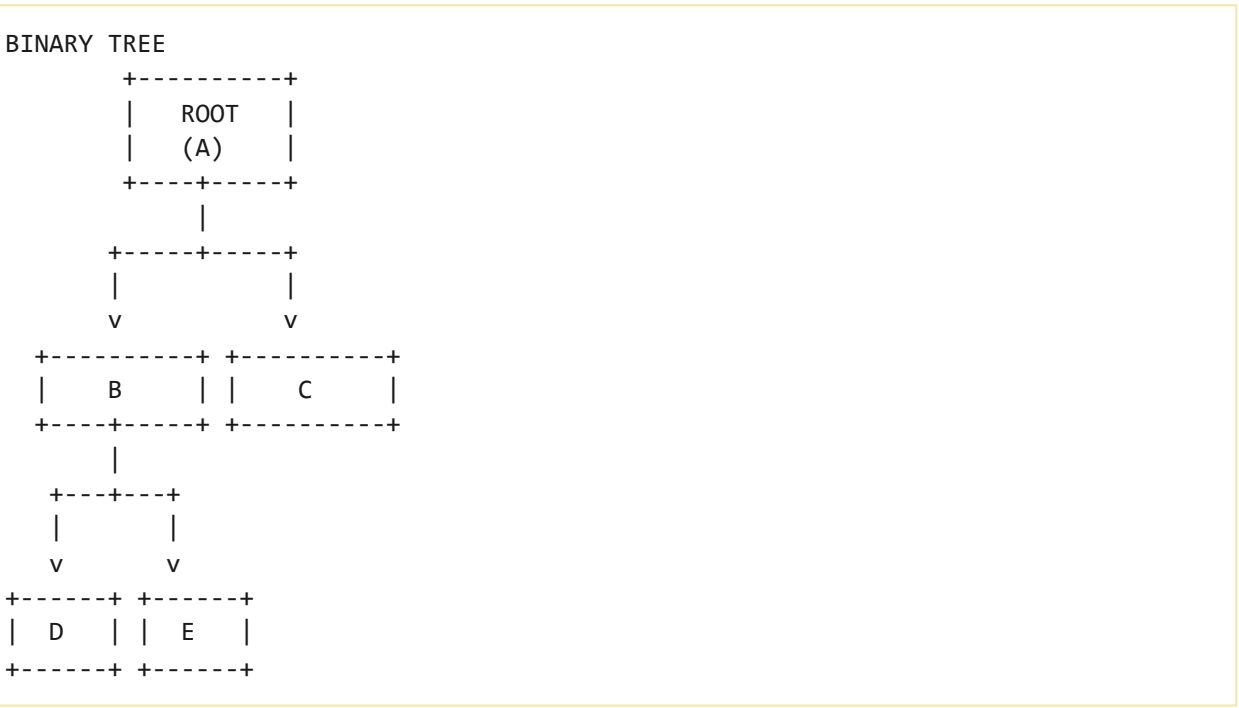
Term	Description
Root	The topmost node of the tree
Node	A single element in the tree
Edge	A connection between two nodes
Parent	A node that has children
Child	A node that has a parent

Siblings	Nodes with the same parent
Leaf	A node with no children
Internal Node	A node with at least one child
Depth	Number of edges from root to a node
Height	Number of edges from root to deepest leaf
Degree	Number of children of a node
Subtree	A tree formed by a node and its descendants

Topic 2: Binary Tree

2.1 What is a Binary Tree?

A **binary tree** is a tree in which **each node has at most two children**, referred to as the **left child** and **right child**.



2.2 Types of Binary Trees

Type	Description
Full Binary Tree	Every node has 0 or 2 children
Complete Binary Tree	All levels filled except possibly the last, which is filled left to right
Perfect Binary Tree	All internal nodes have 2 children and all leaves are at the same level
Balanced Binary Tree	Height difference between left and right subtrees is at most 1 (e.g., AVL Tree)

Degenerate Tree	Each node has only one child (skewed tree, behaves like a linked list)

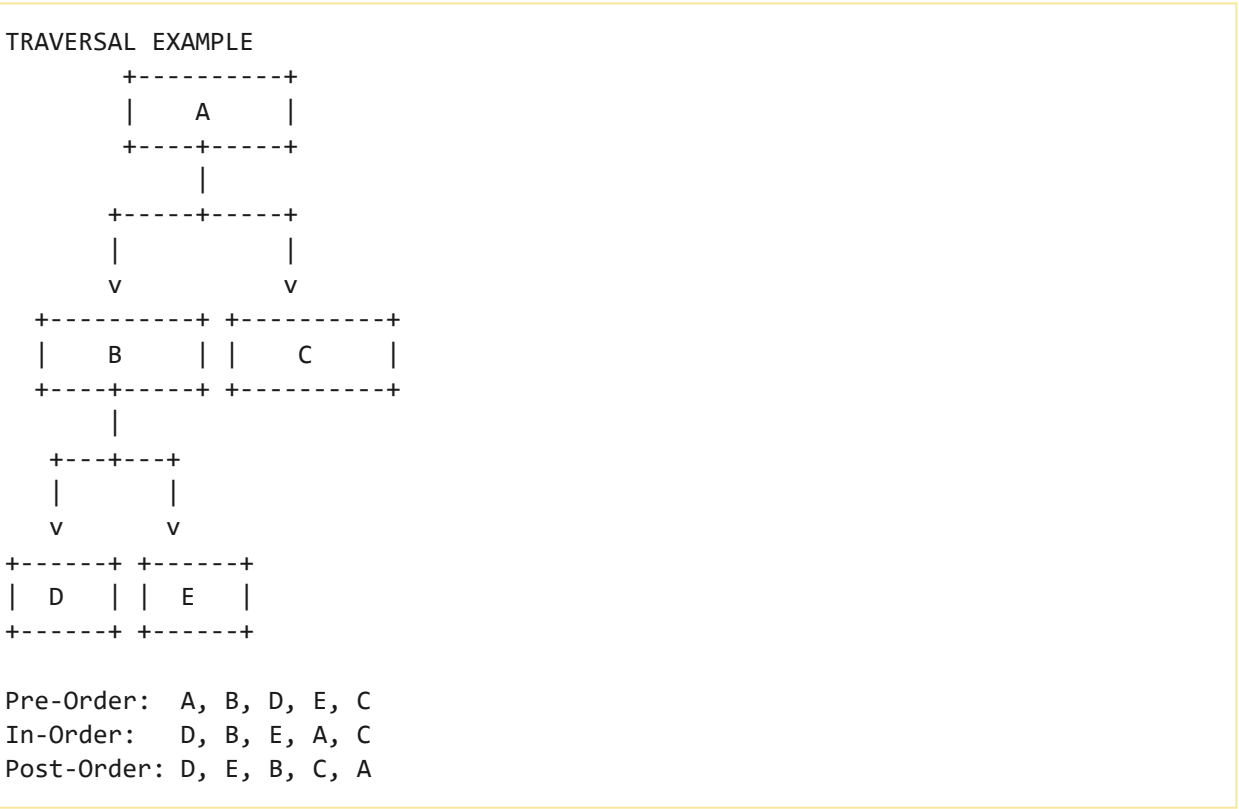
2.3 Node Structure

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Topic 3: Tree Traversals

3.1 Types of Traversals

Method	Order	Description
Pre-Order	Root → Left → Right	Visit root first, then left subtree, then right subtree
In-Order	Left → Root → Right	Visit left subtree, then root, then right subtree
Post-Order	Left → Right → Root	Visit left subtree, then right subtree, then root



3.2 Pre-Order Traversal

Algorithm

```
1. Visit/Process the root node
2. Traverse the left subtree (Pre-Order)
3. Traverse the right subtree (Pre-Order)
```

Code

```
void preOrder(struct Node* node) {
    if (node != NULL) {
        printf("%d ", node->data);
        preOrder(node->left);
        preOrder(node->right);
    }
}
```

3.3 In-Order Traversal

Algorithm

```
1. Traverse the left subtree (In-Order)
2. Visit/Process the root node
3. Traverse the right subtree (In-Order)
```

Code

```
void inOrder(struct Node* node) {
    if (node != NULL) {
        inOrder(node->left);
        printf("%d ", node->data);
        inOrder(node->right);
    }
}
```

3.4 Post-Order Traversal

Algorithm

```
1. Traverse the left subtree (Post-Order)
2. Traverse the right subtree (Post-Order)
3. Visit/Process the root node
```

Code

```
void postOrder(struct Node* node) {
    if (node != NULL) {
        postOrder(node->left);
        postOrder(node->right);
        printf("%d ", node->data);
    }
}
```

3.5 Complete Traversal Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

void preOrder(struct Node* node) {
    if (node != NULL) {
        printf("%d ", node->data);
        preOrder(node->left);
        preOrder(node->right);
    }
}
```

```
void inOrder(struct Node* node) {
    if (node != NULL) {
        inOrder(node->left);
        printf("%d ", node->data);
        inOrder(node->right);
    }
}

void postOrder(struct Node* node) {
    if (node != NULL) {
        postOrder(node->left);
        postOrder(node->right);
        printf("%d ", node->data);
    }
}

int main() {
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Pre-Order: ");  preOrder(root);  printf("\n");
    printf("In-Order: ");   inOrder(root);   printf("\n");
    printf("Post-Order: "); postOrder(root);  printf("\n");
    return 0;
}
```

Output: Pre-Order: 1 2 4 5 3 | In-Order: 4 2 5 1 3 | Post-Order: 4 5 2 3 1

Topic 4: Binary Search Tree (BST)

4.1 BST Property

A **Binary Search Tree (BST)** is a binary tree where for every node:

- All elements in the **left subtree** are **less than** the node
- All elements in the **right subtree** are **greater than** the node

```
BINARY SEARCH TREE
        +----------+
        |    50    |
        +----+-----+
             |
       +-----+-----+
       |           |
       v           v
  +----------+ +----------+
  |    30    | |    70    |
  +----+-----+ +----+-----+
       |            |
   +---+---+    +---+---+
   |       |    |       |
   v       v    v       v
+------+ +------+ +------+ +------+
|  20  | |  40  | |  60  | |  80  |
+------+ +------+ +------+ +------+

BST Property:
  Left subtree: All values < 50
  Right subtree: All values > 50
```

4.2 BST Search

Algorithm

```
1. Start from the root node
2. If root is NULL, return NULL
3. If value == root->data, return root
4. If value < root->data, search in left subtree
5. If value > root->data, search in right subtree
```

Code

```
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);
    return search(root->right, key);
}
```

4.3 BST Insertion

Algorithm

```
1. If root is NULL, create a new node and return it
2. If value < root->data, insert into left subtree
3. If value > root->data, insert into right subtree
4. If value == root->data, return (duplicate)
```

Code

```
struct Node* insert(struct Node* root, int key) {
    if (root == NULL)
        return createNode(key);
    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);
    return root;
}
```

4.4 BST Deletion

Three Cases

Case	Description	Action

Case 1	Leaf node (no children)	Remove the node directly
Case 2	Node with one child	Replace node with its child
Case 3	Node with two children	Find inorder successor, replace data, delete successor

Code

```
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        struct Node* temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}
```

Topic 5: Sorting Techniques

5.1 Selection Sort

Selection sort repeatedly **selects the smallest element** from the unsorted part and places it at the beginning.

```
Selection Sort Example:
Array: [64, 25, 12, 22, 11]
Step 1: Find min(11), swap with 64 -> [11, 25, 12, 22, 64]
Step 2: Find min(12), swap with 25 -> [11, 12, 25, 22, 64]
Step 3: Find min(22), swap with 25 -> [11, 12, 22, 25, 64]
Step 4: Find min(25), swap with 25 -> [11, 12, 22, 25, 64]
Result: [11, 12, 22, 25, 64]
```

Algorithm

```
1. Set min_idx = i for each position i
2. Find the smallest element in the unsorted portion (i to n-1)
3. Swap the smallest element with arr[i]
4. Repeat for all positions
```

Code

```
void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

Complexity: $O(n^2)$ in all cases

5.2 Bubble Sort

Bubble sort works by repeatedly **swapping adjacent elements** if they are in the wrong order.

```
Bubble Sort Example:
Array: [64, 34, 25, 12, 22, 11, 90]
Pass 1: 34, 25, 12, 22, 11, 64, 90
Pass 2: 25, 12, 22, 11, 34, 64, 90
Pass 3: 12, 22, 11, 25, 34, 64, 90
Pass 4: 12, 11, 22, 25, 34, 64, 90
Pass 5: 11, 12, 22, 25, 34, 64, 90
Pass 6: 11, 12, 22, 25, 34, 64, 90
Result: [11, 12, 22, 25, 34, 64, 90]
```

Algorithm

```
1. For each pass i from 0 to n-2:
2.   For each j from 0 to n-i-2:
3.     If arr[j] > arr[j+1], swap them
4. Repeat until no swaps needed
```

Code

```
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Complexity: O(n) best, $O(n^2)$ average/worst

5.3 Merge Sort

Merge sort is a **divide and conquer** algorithm that divides the array into two halves, recursively sorts them, and then merges them.

```
Merge Sort Example:
        [38, 27, 43, 3, 9, 82, 10]
                /              \
        [38, 27, 43, 3]    [9, 82, 10]
          /        \          /     \
     [38, 27]   [43, 3]   [9, 82]  [10]
      /    \     /    \     /   \     |
    [38]  [27] [43]  [3]  [9]  [82] [10]
      \    /     \    /     \   /     |
    [27, 38]   [3, 43]   [9, 82]    [10]
          \        /          \      /
       [3, 27, 38, 43]    [9, 10, 82]
                \            /
          [3, 9, 10, 27, 38, 43, 82]
```

Algorithm

1. Divide the array into two halves
2. Recursively sort each half
3. Merge the two sorted halves

Code

```
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++) L[i] = arr[left + i];
    for (j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    i = 0; j = 0; k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
```

```
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Complexity: O(n log n) in all cases

5.4 Radix Sort

Radix sort sorts elements by individual digits, from least significant to most significant digit.

Algorithm

```
1. Find the maximum element to determine number of digits
2. For each digit position (units, tens, hundreds...):
   a. Use counting sort to sort elements by that digit
3. Repeat until all digit positions are processed
```

Code

```
int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max) max = arr[i];
    return max;
}

void countingSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};
    for (int i = 0; i < n; i++)
```

```
        count[(arr[i] / exp) % 10]++;
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}
```

Complexity: O(d × n) where d is number of digits

Topic 6: Searching Techniques

6.1 Linear Search

Linear search checks each element one by one until the target is found.

```
Linear Search Example:
Array: [10, 23, 45, 70, 11, 15, 99]  Target: 70
Step 1: arr[0]=10 != 70
Step 2: arr[1]=23 != 70
Step 3: arr[2]=45 != 70
Step 4: arr[3]=70 -> FOUND at index 3
```

Algorithm

```
1. For i = 0 to n-1:
2.   If arr[i] == key, return i
3. Return -1 (not found)
```

Code

```
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
```

Complexity: O(n)

6.2 Binary Search

Binary search works on **sorted arrays** by repeatedly dividing the search interval in half.

```
Binary Search Example:
Sorted Array: [11, 12, 22, 25, 34, 64, 90]  Target: 25
```

```
Step 1: low=0, high=6, mid=3 -> arr[3]=25 -> FOUND!

Target: 15
Step 1: low=0, high=6, mid=3 -> arr[3]=25 > 15
Step 2: low=0, high=2, mid=1 -> arr[1]=12 < 15
Step 3: low=2, high=2, mid=2 -> arr[2]=22 > 15
Step 4: low=2, high=1 -> NOT FOUND
```

Algorithm (Iterative)

```
1. Set low = 0, high = n - 1
2. While low <= high:
   a. mid = (low + high) / 2
   b. If arr[mid] == key, return mid
   c. If arr[mid] < key, low = mid + 1
   d. Else, high = mid - 1
3. Return -1 (not found)
```

Code (Iterative)

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Code (Recursive)

```
int binarySearchRec(int arr[], int low, int high, int key) {
    if (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key)
            return binarySearchRec(arr, mid + 1, high, key);
        return binarySearchRec(arr, low, mid - 1, key);
    }
```

```
    return -1;
}
```

Search Comparison

Feature	Linear Search	Binary Search
Requirement	None	Sorted array
Time Complexity	O(n)	O(log n)
Best Case	O(1)	O(1)
Worst Case	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Use Case	Small/Unsorted arrays	Large/Sorted arrays

Topic 7: Quick Revision - Complexity Comparison

Sorting Algorithms Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Radix Sort	O(d × n)	O(d × n)	O(d × n)	O(n + k)	Yes

Searching Comparison

Technique	Time Complexity	Requirement
Linear Search	O(n)	None
Binary Search	O(log n)	Sorted Array

Tree Traversals

Type	Order
Pre-Order	Root → Left → Right
In-Order	Left → Root → Right
Post-Order	Left → Right → Root

BST Property

- Left Subtree: All values < Root
- Right Subtree: All values > Root

When to Use Which Algorithm

Algorithm	Best Use Case
Selection Sort	Small datasets, simple implementation
Bubble Sort	Educational purposes, nearly sorted data
Merge Sort	Large datasets, stable sort required
Quick Sort	Large datasets, average case performance
Radix Sort	Integer data with fixed digit length
Linear Search	Small or unsorted arrays
Binary Search	Large sorted arrays

Questions

Short Answer Questions (2-5 Marks)

1. What is a tree data structure?
2. What is a binary tree?
3. Define BST (Binary Search Tree).
4. What are the types of tree traversals?
5. What is the difference between a binary tree and a BST?
6. What is linear search? Give its time complexity.
7. What is binary search? Give its time complexity.
8. What is the difference between linear and binary search?
9. What is the time complexity of merge sort?
10. Define: root, leaf, depth, height in a tree.

Long Answer Questions (10 Marks)

1. Explain binary trees and their traversal methods with examples.
2. Explain Binary Search Trees with insertion and deletion operations.
3. Explain tree traversal algorithms (Pre, In, Post) with code and example.
4. Explain selection sort with algorithm, example, and code.
5. Explain bubble sort with algorithm, example, and code.
6. Explain merge sort with algorithm, example, and code.
7. Explain linear and binary search with examples and code.
8. Compare all sorting algorithms with complexity analysis.
9. Explain radix sort with algorithm and code.

Objective Questions (1 Mark)

1. In a binary tree, each node has at most: (a) 1 child (b) 2 children (c) 3 children (d) n children
   Answer: (b)
2. In BST, left subtree values are: (a) Greater (b) Smaller (c) Equal (d) None
   Answer: (b)

3. Time complexity of binary search is: (a) O(n) (b) O(log n) (c) $O(n^2)$ (d) O(1)
   Answer: (b)
4. Merge sort time complexity is: (a) O(n) (b) O(log n) (c) O(n log n) (d) $O(n^2)$
   Answer: (c)
5. Selection sort worst-case complexity is: (a) O(n) (b) O(log n) (c) O(n log n) (d) $O(n^2)$
   Answer: (d)
6. Pre-order traversal is: (a) Root-Left-Right (b) Left-Root-Right (c) Left-Right-Root (d) Root-Right-Left
   Answer: (a)
7. Binary search requires: (a) Unsorted array (b) Sorted array (c) Linked list (d) Stack
   Answer: (b)
8. Which sort is O(n log n) in worst case? (a) Bubble (b) Selection (c) Merge (d) Quick
   Answer: (c)